

Code Explanation

Data Processing Workflow Explanation

This code is a data processing workflow written in Python using the PySpark library. It reads customer and order data from CSV files, cleans the data, creates and updates an order summary table using Slowly Changing Dimension (SCD) Type 2 logic, and finally creates a customer aggregate spend table.

Overview

The code consists of several functions that perform different tasks in the data processing workflow. The main function orchestrates the entire workflow by calling these functions in a specific order.

Step 1: Create a SparkSession Object

The first step is to create a SparkSession object, which is the entry point to any PySpark functionality.

```
def get_spark_session():  
    """Create and return a SparkSession."""  
    return SparkSession.builder  
        .appName("Customer Order Processing")  
        .getOrCreate()
```

Step 2: Read Source Data

The next step is to read the customer and order data from CSV files.

The `read_source_data` function reads the data into DataFrames using the `spark.read.format("csv")` method. It also defines schemas for the customer and order data to ensure better control and performance.

```
customer_schema = StructType([  
    StructField("CustId", StringType(), True),  
    StructField("Name", StringType(), True),  
    StructField("EmailId", StringType(), True),  
    StructField("Region", StringType(), True)  
)  
  
order_schema = StructType([  
    StructField("OrderId", StringType(), True),  
    StructField("ItemName", StringType(), True),  
    StructField("PricePerUnit", DoubleType(), True),  
    StructField("Qty", IntegerType(), True),  
    StructField("Date", DateType(), True),  
    StructField("CustId", StringType(), True)  
)  
  
customer_df = spark.read.format("csv")  
    .option("header", "true")  
    .schema(customer_schema)  
    .load("/Volumes/gen_ai_poc_databrickscoe/sdlc_wizard/customerdata")  
  
order_df = spark.read.format("csv")  
    .option("header", "true")
```

```
.schema(order_schema)
.load("/Volumes/gen_ai_poc_databrickscoe/sdlc_wizard/orderdata")
```

Step 3: Clean the Data

The `clean\_data` function removes null and duplicate records from the customer and order DataFrames.

```
cleaned_customer_df = customer_df.dropDuplicates(["CustId"])
    .filter(col("CustId").isNotNull() &
            col("Name").isNotNull() &
            col("EmailId").isNotNull() &
            col("Region").isNotNull())

cleaned_order_df = order_df.withColumn("TotalAmount", col("PricePerUnit") * col("Qty"))
    .dropDuplicates(["OrderId"])
    .filter(col("OrderId").isNotNull() &
            col("ItemName").isNotNull() &
            col("PricePerUnit").isNotNull() &
            col("Qty").isNotNull() &
            col("Date").isNotNull() &
            col("CustId").isNotNull())
```

Step 4: Create and Update the Order Summary Table

The `create\_order\_summary\_table` function creates and updates the order summary table using SCD Type 2 logic.

It first checks if the order summary table exists. If it doesn't exist, it creates the table with the required columns. If it exists, it calls the `update\_order\_summary\_scd2` function to apply SCD Type 2 logic.

```
if not table_exists:
    joined_df = joined_df.withColumn("IsActive", lit(True))
        .withColumn("StartDate", lit(current_date).cast("date"))
        .withColumn("EndDate", lit(None).cast("date"))

    joined_df.write.format("delta")
        .mode("overwrite")
        .saveAsTable("gen_ai_poc_databrickscoe.sdlc_wizard.ordersummary")
else:
    update_order_summary_scd2(spark, joined_df)
```

Step 5: Update the Order Summary Table using SCD Type 2 Logic

The `update\_order\_summary\_scd2` function updates the order summary table using SCD Type 2 logic.

It adds SCD Type 2 columns to the new data, gets the existing order summary table as a DeltaTable, and merges the new data into the existing table.

```

order_summary_table.alias("target").merge(
    new_data_with_scd.alias("source"),
    """target.CustId = source.CustId AND
    target.OrderId = source.OrderId AND
    target.IsActive = true AND
    (target.Name != source.Name OR
    target.EmailId != source.EmailId OR
    target.Region != source.Region OR
    target.ItemName != source.ItemName OR
    target.PricePerUnit != source.PricePerUnit OR
    target.Qty != source.Qty OR
    target.Date != source.Date OR
    target.TotalAmount != source.TotalAmount)"""
).whenMatchedUpdate(
    set={
        "IsActive": "false",
        "EndDate": lit(current_date).cast("date")
    }
).whenNotMatchedInsert(
    values={
        "CustId": "source.CustId",
        "Name": "source.Name",
        "EmailId": "source.EmailId",
        "Region": "source.Region",
        "OrderId": "source.OrderId",
        "ItemName": "source.ItemName",
        "PricePerUnit": "source.PricePerUnit",
        "Qty": "source.Qty",
        "Date": "source.Date",
        "TotalAmount": "source.TotalAmount",
        "IsActive": "true",
        "StartDate": lit(current_date).cast("date"),
        "EndDate": "null"
    }
).execute()

```

Step 6: Create the Customer Aggregate Spend Table

The `create\_customer\_aggregate\_spend` function creates the customer aggregate spend table.

It aggregates data from the order summary table and writes the aggregated data to the customer aggregate spend table.

```

aggregate_df = spark.sql("""
    SELECT
        Name,
        SUM(TotalAmount) AS TotalAmount,
        Date
    FROM gen_ai_poc_databrickscoe.sdlc_wizard.ordersummary
    WHERE IsActive = true
    GROUP BY Name, Date
""")

aggregate_df.write.format("delta")
    .mode("overwrite")
    .saveAsTable("gen_ai_poc_databrickscoe.sdlc_wizard.customeraggregate_spend")

```

Step 7: Main Function

The `main` function orchestrates the entire data processing workflow by calling the above functions in a specific order.

```
def main():
    """Main function to execute the data processing workflow."""
    spark = get_spark_session()

    # Read source data
    customer_df, order_df = read_source_data(spark)

    # Clean data
    cleaned_customer_df, cleaned_order_df = clean_data(customer_df,
order_df)

    # Create and update the order summary table
    create_order_summary_table(spark, cleaned_customer_df, cleaned_o
rder_df)

    # Create the customer aggregate spend table
    create_customer_aggregate_spend(spark)

    print("Data processing completed successfully.")

if __name__ == "__main__":
    main()
```